CLASE

Introducción a Terraform



Curso de Automatización · Terraform

Resumen · 30 min

¿Qué es Terraform?

Creado por HashiCorp

Herramienta de infraestructura como código lanzada en 2014. HashiCorp hoy forma parte de IBM.

Declarativo

Describes QUÉ infraestructura quieres; Terraform decide CÓMO y en qué orden crearla.

Lenguaje HCL

Se escribe en HCL (HashiCorp Configuration Language): legible, estructurado y reutilizable.

Infraestructura como código (IaC)

Definir servidores, redes y firewalls en archivos de texto, en lugar de crearlos a mano con clics en la consola.



Codificar

La infraestructura queda escrita, versionada y revisable.



Repetir

El mismo código crea entornos idénticos: dev, test, prod.



Cualquier nube

AWS, Azure, Google Cloud y cientos de servicios más.

PAUSA · MEME



Por eso existe la infraestructura como código.

Beneficios de Terraform

Control de versiones

Cambios en Git: historial y vuelta atrás.

Automatización

Menos tareas manuales, despliegues más rápidos.

Colaboración

El equipo revisa la infraestructura como código.

Consistencia

Mismos entornos, sin diferencias manuales.

Escalabilidad

Crecer o reducir cambiando un número.

Reutilización

Variables y módulos para no repetir código.

Independiente de la plataforma

Un mismo lenguaje y flujo; cada plataforma tiene su provider



Declarativo vs imperativo

Declarativo (Terraform)

- «Quiero una EC2 t3.micro con este firewall»
- Describes el resultado final
- Terraform calcula los pasos

Imperativo (script)

- Paso 1: crear la VM
- Paso 2: crear el firewall
- Paso 3: asociarlos
- Tú defines cada paso y su orden

Con Terraform describes el QUÉ; no el CÓMO.

Estado deseado



- Lo que escribes en los .tf es el estado deseado: el plano de tu infraestructura.
- Terraform compara ese plano con lo que existe y aplica solo las diferencias.
- Si ya coincide, no hace nada.

Componentes principales



Terraform Core

El programa (CLI) que lee el código y coordina todo.



Providers

Plugins para cada plataforma: aws, azurerm, google...



Resources

Lo que se crea: una EC2, un Security Group...



Data sources

Consultan algo que ya existe, sin crearlo.



State

Archivo que relaciona el código con los recursos reales.



Modules

Bloques de código reutilizables.

Terraform y otras herramientas

IaC · similares

- Terraform / OpenTofu
- AWS CloudFormation
- Azure Bicep
- Pulumi

Configuración · complementarias

- Ansible
- Chef
- Puppet
- Instalan paquetes y configuran servidores

Terraform crea la VM → **Ansible** la configura por dentro

HCL

El lenguaje de Terraform



Curso de Automatización · Terraform

Resumen · 30 min

Sintaxis de un bloque

```
sintaxis
tipo_de_bloque "etiqueta" "etiqueta" {
  argumento = valor
  otro      = expresión
}
```

- Los comentarios empiezan con #
- Los archivos terminan en .tf

Palabra clave

Empieza con resource, data, variable, output...

Tiene etiquetas

Entre comillas: tipo y nombre.

Llaves { }

Dentro van los argumentos: nombre = valor.

Anatomía de un resource

```
resource "aws_instance" "demo" {
```



Dirección: `aws_instance.demo`

- Tipo + nombre identifican al recurso dentro del código.
- Así lo referencian otros bloques: aws_instance.demo.public_ip

Tipos de bloques

Bloque	Para qué sirve	Archivo de la demo
`terraform`	Versión de Terraform y providers	versions.tf
`provider`	Configura la conexión (región)	provider.tf
`variable`	Parámetros de entrada	variables.tf
`data`	Consulta algo que ya existe (AMI, VPC)	data.tf
`resource`	Crea algo (EC2, Security Group)	main.tf, security_group.tf
`output`	Muestra valores al final (IP, ssh)	outputs.tf

Referencias entre bloques

main.tf

```hcl
resource "aws_instance" "demo" {
  ami           = data.aws_ami.amazon_linux.id
  instance_type = var.instance_type
  key_name      = aws_key_pair.demo.key_name
  vpc_security_group_ids = [
    aws_security_group.web_ssh.id
  ]

  tags = {
    Name = "${var.project_name}-vm"
  }
}
```

`var.<nombre>`

Valor de una variable

`data.<tipo>.<nombre>`

Dato consultado

`<tipo>.<nombre>.<atributo>`

Atributo de otro recurso

`"${ ... }"`

Insertar un valor en un texto

Variables y outputs

variables.tf

```
variable "instance_type" {
  description = "Tipo de EC2"
  type        = string
  default     = "t3.micro"
}
```

outputs.tf

```
output "public_ip" {
  description = "IP pública"
  value       = aws_instance.demo.public_ip
}
```

terraform.tfvars

```
instance_type = "t3.small"
```

- Variables: cambiar valores sin tocar el código
- terraform.tfvars o -var=... sobrescriben el default
- Outputs: datos útiles al terminar (IP, comando SSH)

Guía de estilo HCL

- Comentarios con #
- Nombres con guiones_bajos (snake_case)
- 2 espacios de sangría por nivel
- Alinear los = de líneas seguidas
- Líneas en blanco entre grupos de argumentos
- terraform fmt lo hace automáticamente

main.tf

```
resource "aws_key_pair" "demo" {
  key_name   = "demo-key"
  public_key = file("~/.ssh/id_rsa.pub")
}
```

TERMINAL

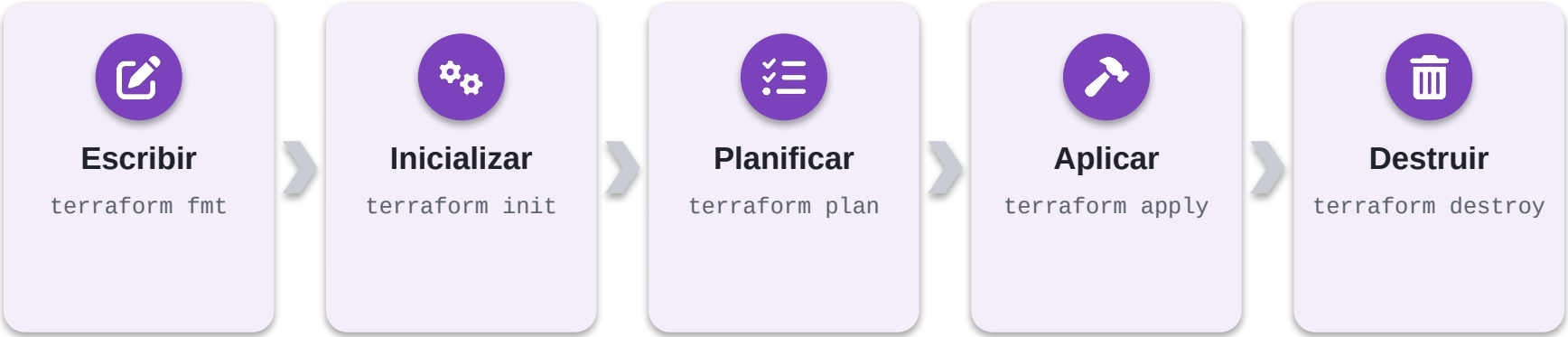
```
$ terraform fmt
main.tf
```

FLUJO

El flujo de trabajo de Terraform

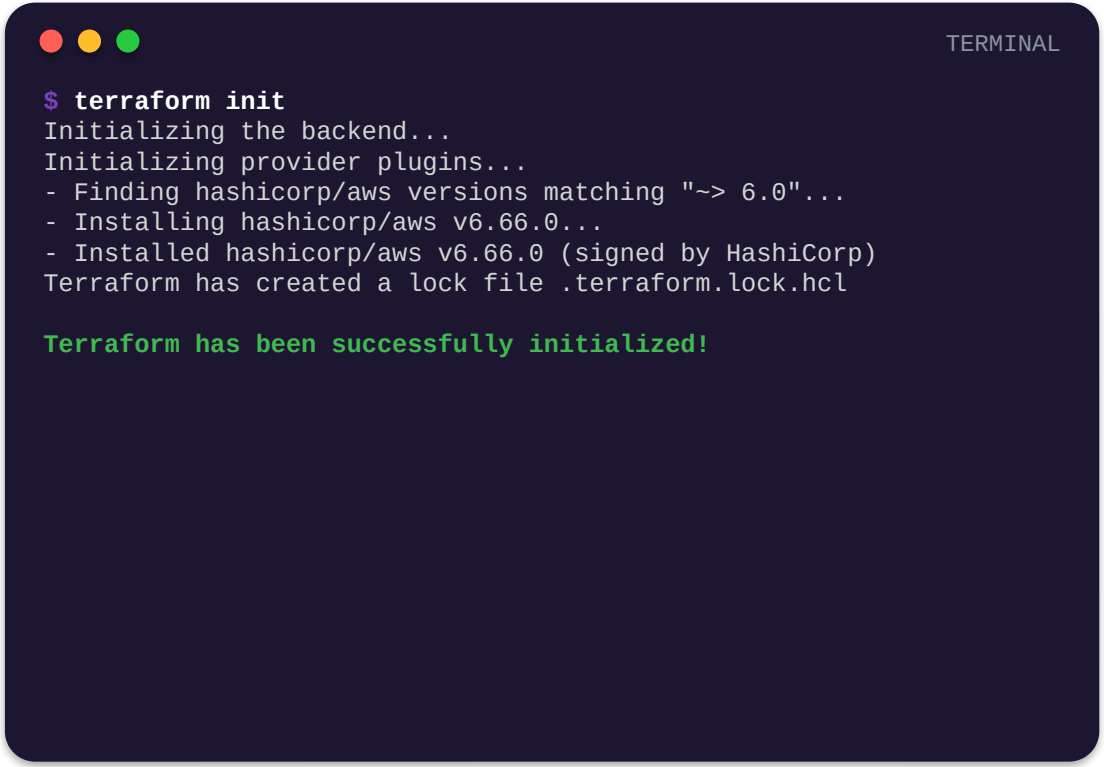


El flujo de trabajo



Patrón de todos los comandos: terraform <subcomando> [opciones]

terraform init



Descarga providers

En la carpeta .terraform/ (no va a Git).

.terraform.lock.hcl

Fija las versiones. Sí va a Git.

¿Cuándo?

Al empezar y al cambiar providers o backend.

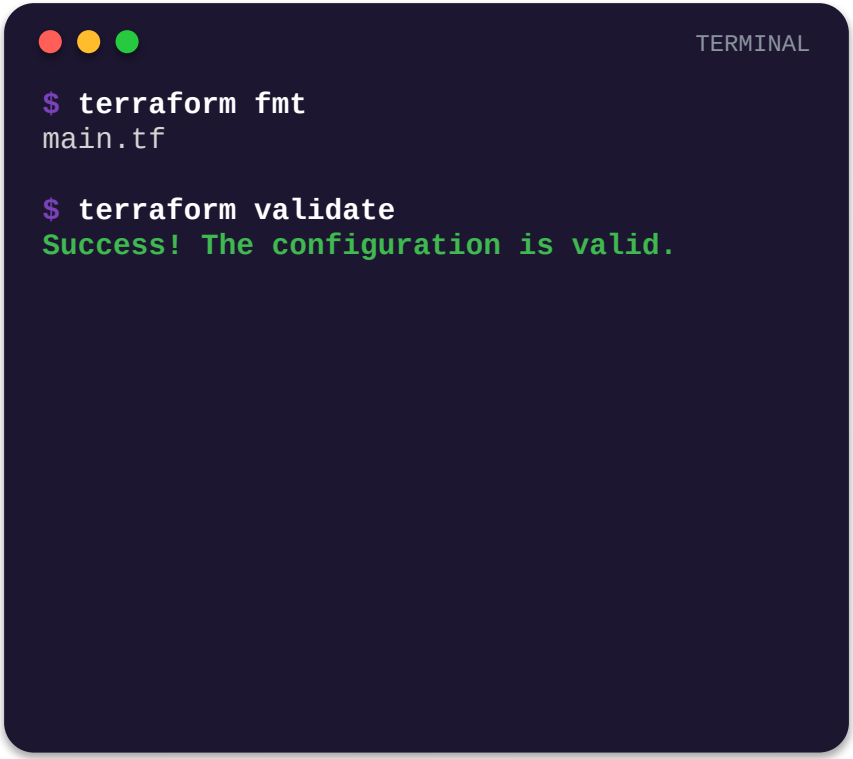
fmt y validate

terraform fmt

- Formatea el código al estilo estándar
- Usa -recursive para subcarpetas

terraform validate

- Revisa sintaxis y referencias
- No habla con AWS: no detecta todo
- Requiere init antes



PAUSA · MEME

EXPECTATIVA	REALIDAD
	
`$ terraform validate` `Success! The configuration is valid.`	`$ terraform apply` `Error: UnauthorizedOperation: You are not authorized...`

validate no habla con AWS: no detecta permisos, cuotas ni límites.

terraform plan

TERMINAL

```
$ terraform plan

Terraform will perform the following actions:

  # aws_instance.demo will be created
  + resource "aws_instance" "demo" {
      + ami           = "ami-0b2c9d1f3edcfd709"
      + instance_type = "t3.micro"
      + public_ip     = (known after apply)
      ...
    }

Plan: 6 to add, 0 to change, 0 to destroy.
```

- Simulacro (dry-run): no cambia nada
- Muestra qué creará, cambiará o destruirá
- (known after apply): lo sabrá AWS al crear
- Léelo siempre antes de aplicar

Leer el plan

Símbolo	Significado	Ejemplo
`+`	Crear	Recurso nuevo en el código
`~`	Modificar sin recrear (in-place)	Cambiar un tag
`-`	Destruir	Recurso borrado del código
`-/+`	Destruir y crear de nuevo	Cambiar la AMI: forces replacement

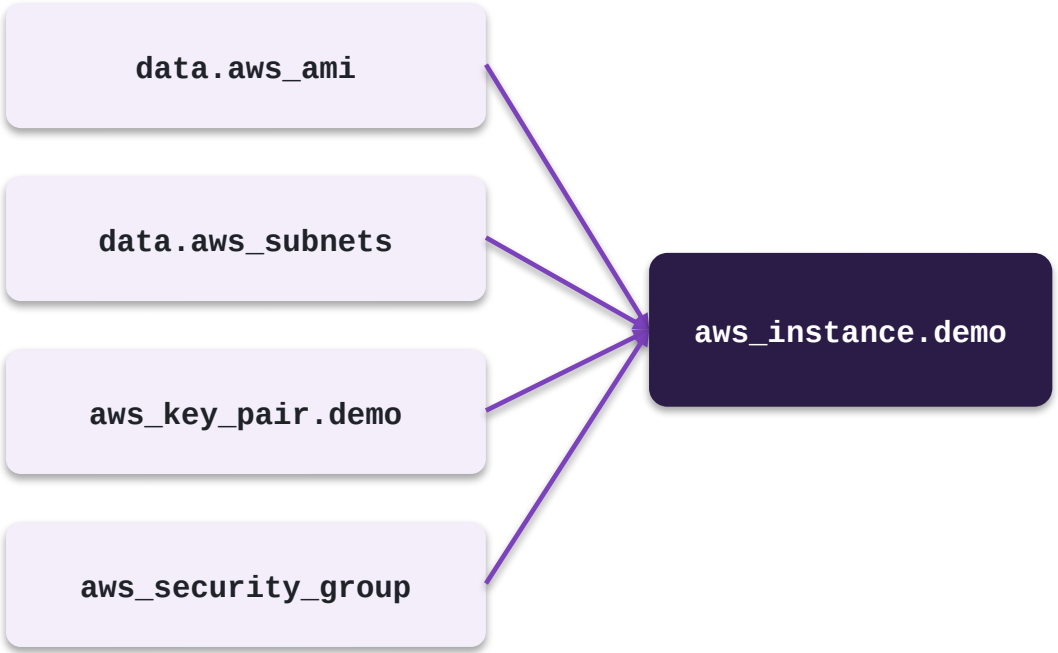
Cuidado con -/+ : el recurso se pierde y se crea uno nuevo (nueva IP, disco vacío).

PAUSA · MEME



Lee siempre el plan: -/+ significa destruir y volver a crear.

Orden de creación



- Terraform construye un grafo con las referencias
- El orden de los bloques en los archivos no importa
- Lo independiente se crea en paralelo (10 a la vez)
- depends_on: dependencia manual, si hace falta

terraform apply

TERMINAL

```
$ terraform apply
Plan: 6 to add, 0 to change, 0 to destroy.
Do you want to perform these actions?
  Enter a value: yes

aws_key_pair.demo: Creating...
aws_instance.demo: Creation complete after 13s

Apply complete! Resources: 6 added, 0 changed, 0 destroyed.

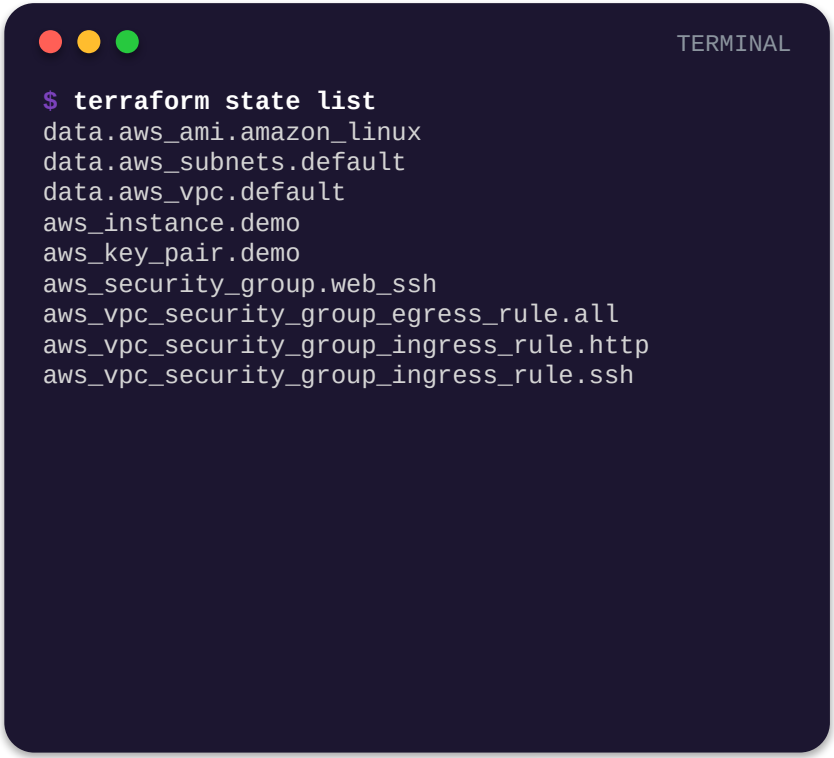
Outputs:
public_ip   = "3.91.10.20"
ssh_command = "ssh -i ~/.ssh/id_rsa ec2-user@3.91.10.20"
```

- Crea los recursos de verdad
- Pide confirmar: escribe yes
- -auto-approve omite la confirmación (con cuidado)
- Si algo falla, no hay rollback: lo creado se queda

El estado: terraform.tfstate

Archivo donde Terraform anota qué recursos creó y sus IDs reales en AWS.

- Se crea en el primer apply
- No se edita a mano
- No se sube a Git: puede tener datos sensibles
- .backup: copia del estado anterior
- En equipos se guarda en remoto (S3, HCP Terraform)



PAUSA · MEME

Nadie:

Absolutamente nadie:

Yo editando terraform.tfstate a mano:



El estado nunca se edita a mano: usa los comandos terraform state.

Modificar la infraestructura

~ in-place

TERMINAL

```
~ resource "aws_instance" "demo" {
    ~ instance_type = "t3.micro" -> "t3.small"
```

Se modifica el mismo recurso. La instancia se detiene y arranca, pero conserva su disco y su ID.

-/+ replace

TERMINAL

```
-/+ resource "aws_instance" "demo" {
      ~ ami = "ami-0b2c..." -> "ami-0f1e..."
        # forces replacement
```

Algunos argumentos no se pueden cambiar en caliente: Terraform destruye la EC2 y crea otra (nueva IP, disco nuevo).

terraform destroy

TERMINAL

```
$ terraform destroy

  # aws_instance.demo will be destroyed
  - resource "aws_instance" "demo" {
      ...

Plan: 0 to add, 0 to change, 6 to destroy.
Do you really want to destroy all resources?
  Enter a value: yes

Destroy complete! Resources: 6 destroyed.
```

Irreversible

- Borra todo lo gestionado por Terraform
- Lee el plan antes de confirmar
- Uno solo: -target=aws_instance.demo
- Siempre destroy al terminar la clase: evita costos

PAUSA · MEME



Al terminar la práctica: terraform destroy.

Estructura del proyecto

árbol

```
clase-terraform/
├── versions.tf
├── provider.tf
├── variables.tf
├── data.tf
├── security_group.tf
├── main.tf
├── outputs.tf
├── terraform.tfvars.example
├── .gitignore
└── README.md
```

Terraform lee y combina TODOS los .tf de la carpeta: los nombres de archivo son una convención para ordenarse.

Archivos que crea Terraform

- **`.terraform/`** providers · no a Git
- **`.terraform.lock.hcl`** sí a Git
- **`terraform.tfstate`** no a Git
- **`*.tfvars`** no a Git (valores reales)

Comandos esenciales

Comando	Para qué sirve
`terraform init`	Preparar la carpeta y descargar providers
`terraform fmt`	Formatear el código
`terraform validate`	Revisar la sintaxis
`terraform plan`	Ver qué va a cambiar (simulacro)
`terraform apply`	Crear o modificar los recursos
`terraform output`	Ver los outputs (IP, comando SSH)
`terraform state list`	Listar los recursos del estado
`terraform destroy`	Eliminar todo lo creado

RESUMEN



Terraform crea infraestructura como código, de forma declarativa.



Se escribe en HCL: bloques resource, data, variable, output...



Los providers conectan con cada plataforma, como AWS.



Flujo: init → plan → apply → destroy.



El estado (.tfstate) une el código con los recursos reales.



Lee siempre el plan antes de aplicar: + ~ - y -/+.